

Sprint 1 AI Usage Log

Course: SOEN 341 - Software Process

Project: CareerConnect - Job Search and Application Tracking Platform

Sprint: Sprint 1

Responsible Person: Brendan Kelly

Task 1: Finding Authentication Endpoints

Purpose of AI Use:

I used the AI to inspect the CareerContact backend and identify HTTP endpoints related to authentication that a React frontend could use.

Chat Link or Prompt/Response:

Prompt: "Looking for a web API for authentication being served somewhere in this code base. Find any HTTP endpoints. Looking for authentication related endpoints specifically a React frontend might use."

AI-Suggested Content:

The AI identified the Auth0 and Spring Security endpoints:

- `GET /oauth2/authorization/okta` starts the login flow.
- `GET /login/oauth2/code/okta` is the OAuth callback.
- `GET /api/v1/cc/security/redirect` completes post-login processing.
- `POST /api/v1/careercontact/logout` handles logout.
- User, employee, and account-management endpoints are defined in `SecurityController`.

Validation:

I reviewed the endpoint annotations in `SecurityController.java` and the OAuth2 and logout configuration in `SecurityConfig.java`.

Decision:

Accepted.

Reflection:

The AI helped locate the authentication routes and distinguish browser redirects from endpoints that a React frontend can call as API requests.

Task 2: Explaining Endpoint Testing

Purpose of AI Use:

I used the AI to determine whether the endpoints could be tested from a terminal or required Postman.

Chat Link or Prompt/Response:

Prompt: "Can I test these in the terminal or do I need postman or something else? Where in the code are these located?"

AI-Suggested Content:

The AI explained that `curl` can test the API endpoints and that Postman is optional. It also explained that the interactive Auth0 login flow is best opened in a browser and identified the controller and security configuration source files.

Validation:

I compared the suggested commands with the configured backend URL and the request mappings in the source code.

Decision:

Accepted.

Reflection:

The AI provided a practical way to test the API without requiring an additional tool and clarified which routes require browser interaction.

Task 3: Documenting the API

Purpose of AI Use:

I used the AI to determine whether generated API documentation already existed and to create documentation for the authentication-related endpoints.

Chat Link or Prompt/Response:

Prompt: "Do we have generated documentation for these endpoints already?" and "Go ahead and add documentation for the endpoints to the repo in a suitable location."

AI-Suggested Content:

The AI found no Swagger/OpenAPI configuration, generated API specification, or Postman collection. It created `docs/API.md` to document the local URLs,

authentication flow, endpoint behavior, frontend examples, cookies, source locations, and current security behavior.

Validation:

I reviewed the endpoint documentation against `SecurityController.java`, `SecurityConfig.java`, and the Auth0 service classes.

Decision:

Modified before use.

Reflection:

The AI made the existing backend routes easier to discover and provided a single location for frontend developers to reference.

Task 4: Documenting JSON Request and Response Fields

Purpose of AI Use:

I used the AI to explicitly document the fields accepted and returned by endpoints that use JSON.

Chat Link or Prompt/Response:

Prompt: "If any of these endpoints return JSON the fields they return should be explicitly listed as well."

AI-Suggested Content:

The AI inspected the request and response model classes and documented the fields for `UserInfoResponseModel`, `EmployeeResponseModel`, `UserRequestModel`, `EmployeeRequestModel`, and authentication errors in `docs/API.md`.

Validation:

I reviewed the model classes and verified the documented field names against their Java properties and Jackson serialization names.

Decision:

Accepted.

Reflection:

Listing the JSON fields makes the documentation more useful for implementing the React frontend and preparing test requests.

Task 5: Restructuring Endpoint Documentation

Purpose of AI Use:

I used the AI to make the endpoint documentation easier to read and more similar to a Doxygen-style reference.

Chat Link or Prompt/Response:

Prompt: "If the table format doesn't work don't bother keeping it. I think it would be better to list the HTTP request, what it does, takes and what it returns all at once doxygen style."

AI-Suggested Content:

The AI restructured `docs/API.md` so every endpoint has a request, purpose, body, and returns section. Each JSON response includes its fields, while redirect and empty-body responses are identified separately.

Validation:

I reviewed the final documentation for all authentication, user/account, and employee/reviewer routes.

Decision:

Accepted.

Reflection:

The Doxygen-style structure presents each endpoint as a complete reference entry and avoids requiring the reader to combine information from separate tables.